

Socket Programming in C — TCP, UDP, DNS, Timers

1. Common Headers

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <errno.h>
#include <arpa/inet.h>
#include <netinet/in.h>
#include <sys/socket.h>
#include <sys/types.h>
#include <netdb.h>      // getaddrinfo, DNS
#include <sys/time.h>   // struct timeval, select
#include <signal.h>     // for timers via alarm/setitimer
```

Compile: `gcc file.c -o out` (add `-lpthread` if you use threads).

2. TCP — Server Boilerplate

```
#define PORT 8080
#define BACKLOG 5

int main() {
    int listen_fd, conn_fd;
    struct sockaddr_in server_addr, client_addr;
    socklen_t addr_len = sizeof(client_addr);
    char buf[1024];

    // 1. Create socket
    listen_fd = socket(AF_INET, SOCK_STREAM, 0);
    if (listen_fd < 0) { perror("socket"); exit(1); }

    // Optional: allow quick restart of server
    int opt = 1;
    setsockopt(listen_fd, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt));

    // 2. Bind
    memset(&server_addr, 0, sizeof(server_addr));
    server_addr.sin_family = AF_INET;
    server_addr.sin_addr.s_addr = INADDR_ANY; // 0.0.0.0
    server_addr.sin_port = htons(PORT);

    if (bind(listen_fd, (struct sockaddr*)&server_addr, sizeof(server_addr)) < 0) {
        perror("bind"); exit(1);
    }

    // 3. Listen
    if (listen(listen_fd, BACKLOG) < 0) { perror("listen"); exit(1); }
    printf("Listening on port %d...\n", PORT);

    // 4. Accept loop
    while (1) {
        conn_fd = accept(listen_fd, (struct sockaddr*)&client_addr, &addr_len);
        if (conn_fd < 0) { perror("accept"); continue; }

        printf("Client connected: %s\n", inet_ntoa(client_addr.sin_addr));

        // 5. Read / Write
        ssize_t n = recv(conn_fd, buf, sizeof(buf) - 1, 0);
        if (n > 0) {
            buf[n] = '\0';
            printf("Received: %s\n", buf);
            send(conn_fd, "ACK", 3, 0);
        }

        close(conn_fd);
    }

    close(listen_fd);
    return 0;
}
```

3. TCP — Client Boilerplate

```
#define SERVER_IP "127.0.0.1"
#define PORT 8080

int main() {
    int sock_fd;
    struct sockaddr_in server_addr;
    char buf[1024];

    sock_fd = socket(AF_INET, SOCK_STREAM, 0);
    if (sock_fd < 0) { perror("socket"); exit(1); }

    memset(&server_addr, 0, sizeof(server_addr));
    server_addr.sin_family = AF_INET;
    server_addr.sin_port = htons(PORT);
    inet_pton(AF_INET, SERVER_IP, &server_addr.sin_addr);

    if (connect(sock_fd, (struct sockaddr*)&server_addr, sizeof(server_addr)) < 0) {
        perror("connect"); exit(1);
    }

    send(sock_fd, "Hello Server", 12, 0);
    ssize_t n = recv(sock_fd, buf, sizeof(buf) - 1, 0);
    if (n > 0) { buf[n] = '\0'; printf("Server replied: %s\n", buf); }

    close(sock_fd);
    return 0;
}
```

Key TCP calls: `socket()`, `bind()`, `listen()`, `accept()`, `connect()`, `send()` / `recv()` (or `write()` / `read()`), `close()`.

4. UDP — Server Boilerplate

```
#define PORT 9090

int main() {
    int sock_fd;
    struct sockaddr_in server_addr, client_addr;
    socklen_t addr_len = sizeof(client_addr);
    char buf[1024];

    sock_fd = socket(AF_INET, SOCK_DGRAM, 0); // note SOCK_DGRAM
    if (sock_fd < 0) { perror("socket"); exit(1); }

    memset(&server_addr, 0, sizeof(server_addr));
    server_addr.sin_family = AF_INET;
    server_addr.sin_addr.s_addr = INADDR_ANY;
    server_addr.sin_port = htons(PORT);

    if (bind(sock_fd, (struct sockaddr*)&server_addr, sizeof(server_addr)) < 0) {
        perror("bind"); exit(1);
    }

    printf("UDP server waiting on port %d...\n", PORT);

    while (1) {
        ssize_t n = recvfrom(sock_fd, buf, sizeof(buf) - 1, 0,
                             (struct sockaddr*)&client_addr, &addr_len);

        if (n > 0) {
            buf[n] = '\0';
            printf("Received: %s from %s:%d\n", buf,
                  inet_ntoa(client_addr.sin_addr), ntohs(client_addr.sin_port));

            sendto(sock_fd, "ACK", 3, 0,
                   (struct sockaddr*)&client_addr, addr_len);
        }
    }

    close(sock_fd);
    return 0;
}
```

5. UDP — Client Boilerplate

```
#define SERVER_IP "127.0.0.1"
#define PORT 9090
```

```

int main() {
    int sock_fd;
    struct sockaddr_in server_addr;
    socklen_t addr_len = sizeof(server_addr);
    char buf[1024];

    sock_fd = socket(AF_INET, SOCK_DGRAM, 0);
    if (sock_fd < 0) { perror("socket"); exit(1); }

    memset(&server_addr, 0, sizeof(server_addr));
    server_addr.sin_family = AF_INET;
    server_addr.sin_port = htons(PORT);
    inet_pton(AF_INET, SERVER_IP, &server_addr.sin_addr);

    sendto(sock_fd, "Hello UDP", 9, 0,
           (struct sockaddr*)&server_addr, addr_len);

    ssize_t n = recvfrom(sock_fd, buf, sizeof(buf) - 1, 0, NULL, NULL);
    if (n > 0) { buf[n] = '\0'; printf("Server replied: %s\n", buf); }

    close(sock_fd);
    return 0;
}

```

Key UDP calls: `socket()` with `SOCK_DGRAM`, `bind()` (server only, optional for client), `sendto()`, `recvfrom()`, `close()`.
 No `connect()` / `accept()` / `listen()` needed (connectionless).

6. DNS Resolution — `getaddrinfo()`

Use this instead of the deprecated `gethostbyname()`.

```

void resolve_hostname(const char *hostname, const char *port) {
    struct addrinfo hints, *res, *p;
    int status;
    char ipstr[INET6_ADDRSTRLEN];

    memset(&hints, 0, sizeof(hints));
    hints.ai_family = AF_UNSPEC; // IPv4 or IPv6
    hints.ai_socktype = SOCK_STREAM; // or SOCK_DGRAM

    if ((status = getaddrinfo(hostname, port, &hints, &res)) != 0) {
        fprintf(stderr, "getaddrinfo error: %s\n", gai_strerror(status));
        return;
    }

    for (p = res; p != NULL; p = p->ai_next) {
        void *addr;
        char *ipver;

        if (p->ai_family == AF_INET) {
            struct sockaddr_in *ipv4 = (struct sockaddr_in *)p->ai_addr;
            addr = &(ipv4->sin_addr);
            ipver = "IPv4";
        } else {
            struct sockaddr_in6 *ipv6 = (struct sockaddr_in6 *)p->ai_addr;
            addr = &(ipv6->sin6_addr);
            ipver = "IPv6";
        }

        inet_ntop(p->ai_family, addr, ipstr, sizeof(ipstr));
        printf("%s: %s\n", ipver, ipstr);
    }

    freeaddrinfo(res);
}

// Usage: resolve_hostname("www.example.com", "80");

```

Reverse DNS (IP → hostname): USE `getnameinfo()`.

```

struct sockaddr_in sa;
sa.sin_family = AF_INET;
sa.sin_port = htons(80);
inet_pton(AF_INET, "93.184.216.34", &sa.sin_addr);

char host[NI_MAXHOST];
getnameinfo((struct sockaddr*)&sa, sizeof(sa), host, sizeof(host), NULL, 0, 0);
printf("Hostname: %s\n", host);

```

Tip: In real client code, replace the manual `inet_pton` step in your TCP/UDP client with `getaddrinfo()` output so your client works with domain names, not just IPs.

7. Timers — Three Common Approaches

(a) `select()` with a timeout — best for socket read timeouts

```
struct timeval timeout;
fd_set read_fds;

FD_ZERO(&read_fds);
FD_SET(sock_fd, &read_fds);

timeout.tv_sec = 5;    // 5 second timeout
timeout.tv_usec = 0;

int ret = select(sock_fd + 1, &read_fds, NULL, NULL, &timeout);
if (ret == -1) {
    perror("select");
} else if (ret == 0) {
    printf("Timeout! No data received in 5 seconds.\n");
} else {
    if (FD_ISSET(sock_fd, &read_fds)) {
        // socket has data ready to read
        recv(sock_fd, buf, sizeof(buf), 0);
    }
}
```

(b) `setsockopt()` with `SO_RCVTIMEO` — simplest per-socket timeout

```
struct timeval tv;
tv.tv_sec = 3;
tv.tv_usec = 0;
setsockopt(sock_fd, SOL_SOCKET, SO_RCVTIMEO, &tv, sizeof(tv));

// recv() will now return -1 with errno == EWOULDBLOCK/EAGAIN after 3s
ssize_t n = recv(sock_fd, buf, sizeof(buf), 0);
if (n < 0 && (errno == EAGAIN || errno == EWOULDBLOCK)) {
    printf("recv timed out\n");
}
```

(c) `alarm()` + `SIGALRM` — for timing out blocking calls / retransmission timers

```
#include <signal.h>

volatile sig_atomic_t timed_out = 0;

void alarm_handler(int sig) {
    timed_out = 1;
}

int main() {
    signal(SIGALRM, alarm_handler);

    alarm(5); // fire SIGALRM in 5 seconds
    ssize_t n = recv(sock_fd, buf, sizeof(buf), 0); // gets interrupted (EINTR) if it fires
    alarm(0); // cancel pending alarm if recv succeeded first

    if (timed_out) {
        printf("Operation timed out\n");
    }
}
```

(d) `setitimer()` — repeating/periodic timer (e.g. resend every N seconds)

```
struct itimerval timer;
timer.it_value.tv_sec = 1;    // first expiry after 1s
timer.it_value.tv_usec = 0;
timer.it_interval.tv_sec = 1; // repeat every 1s after that
timer.it_interval.tv_usec = 0;

signal(SIGALRM, alarm_handler);
setitimer(ITIMER_REAL, &timer, NULL);
```

Use this for **retransmission logic** on top of UDP (since UDP is unreliable and has no built-in ACK/retry).

8. Practice Questions

TCP 1. Write a TCP echo server that handles one client at a time, echoing back whatever it receives, until the client sends "quit". 2. Extend the echo server to handle **multiple simultaneous clients** using `fork()` (one child process per connection). 3. Redo Q2 using `select()` instead of `fork()`, so a single process multiplexes all client sockets. 4. Write a TCP client that sends a file to the server, and a server that saves the received bytes to disk (handle partial reads/writes with a loop). 5. Add a 5-second `recv()` timeout to your TCP client using `SO_RCVTIMEO`. If the server doesn't respond in time, print an error and exit cleanly.

UDP 6. Write a UDP client-server pair implementing a simple "ping" — client sends a timestamp, server echoes it back, client computes round-trip time. 7. Modify Q6 so that if no reply arrives within 2 seconds, the client **retransmits** the ping up to 3 times before giving up (use `alarm()` or `setitimer()`). 8. Build a simple UDP-based chat: multiple clients send messages to a server, which broadcasts each message to all other known clients (server must track client addresses in a list/array). 9. Implement a basic **reliable-delivery layer over UDP**: add sequence numbers to your packets, have the receiver send ACKs, and have the sender retransmit unACKed packets after a timeout.

DNS 10. Write a program that takes a hostname as a command-line argument and prints all its IPv4 and IPv6 addresses using `getaddrinfo()`. 11. Write a program that takes an IP address and prints the corresponding hostname using `getnameinfo()`. 12. Modify your TCP client (Q1) so instead of hardcoding an IP, it accepts a **hostname** and resolves it via `getaddrinfo()` before connecting.

Timers / combined scenarios 13. Write a TCP server that disconnects any client that has been idle (no data sent) for more than 10 seconds, using `select()`'s timeout to detect inactivity. 14. Simulate a basic **DNS resolver with caching and timeout**: resolve a hostname, cache the result with a 30-second expiry (timestamp check), and re-resolve automatically once expired. 15. Build a mini "reliable file transfer over UDP" tool: split a file into numbered chunks, send them with a retransmit timer per chunk (like Q9), and reassemble on the receiver side, printing total transfer time. 16. Combine everything: write a client that (a) resolves a hostname via DNS, (b) connects over TCP with a connect-timeout using `select()` on a non-blocking socket, (c) falls back to trying the next resolved address if the first times out.

Suggested order to tackle them

Start with Q1–Q2 (TCP basics) → Q6 (UDP basics) → Q10 (DNS) → Q5/Q7 (timeouts) → Q3, Q8, Q9 (concurrency + reliability) → Q13–Q16 (integration).